

torch.fft.rfftn#

<https://pytorch.org/docs/stable/generated/torch.fft.rfftn.html>

Description:

Computes the N-dimensional discrete Fourier transform of real input. The FFT of a real signal is Hermitian-symmetric, $X[i_1, \dots, i_n] = \text{conj}(X[-i_1, \dots, -i_n])$ so the full `fftn()` output contains redundant information. `rfftn()` instead omits the negative frequencies in the last dimension. Supports `torch.half` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

input (Tensor) – the input tensor

Function Signature

```
torch.fft.rfftn(input, s=None, dim=None, norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> t = torch.rand(10, 10)
>>> rfftn = torch.fft.rfftn(t)
>>> rfftn.size()
torch.Size([10, 6])
```

```
>>> fftn = torch.fft.fftn(t)
>>> torch.testing.assert_close(fftn[..., :6], rfftn,
                                check_stride=False)
```

```
>>> two_ffts = torch.fft.fft(torch.fft.rfft(t, dim=1), dim=0)
>>> torch.testing.assert_close(rfftn, two_ffts,
                                check_stride=False)
```

Notes & Warnings

NOTE

Note Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the N-dimensional discrete Fourier transform of real input.

The FFT of a real signal is Hermitian-symmetric, $X[i_1, \dots, i_n] = \text{conj}(X[-i_1, \dots, -i_n])$ so the full `fftn()` output contains redundant information. `rfftn()` instead omits the negative frequencies in the last dimension.

Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

`input (Tensor)` – the input tensor

`s (Tuple[int], optional)` – Signal size in the transformed dimensions. If given, each dimension `dim[i]` will either be zero-padded or trimmed to the length `s[i]` before computing the real FFT. If a length `-1` is specified, no padding is done in that dimension. Default: `s = [input.size(d) for d in dim]`

`dim (Tuple[int], optional)` – Dimensions to be transformed. Default: all dimensions, or the

last len(s) dimensions if s is given.

norm (str, optional) –

Normalization mode. For the forward transform (rfftn()), these correspond to:

"forward" - normalize by 1/n

"backward" - no normalization

"ortho" - normalize by 1/sqrt(n) (making the real FFT orthonormal)

Where $n = \text{prod}(s)$ is the logical FFT size. Calling the backward transform (irfftn()) with the same normalization mode will apply an overall normalization of 1/n between the two transforms. This is required to make irfftn() the exact inverse.

Default is "backward" (no normalization).

out (Tensor, optional) – the output tensor.

Compared against the full output from fftn(), we have all elements up to the Nyquist frequency.

The discrete Fourier transform is separable, so rfftn() here is equivalent to a combination of fft() and rfft():